

Programmation Fonctionnelle

TD n° 9 : Entrées-Sorties

Pour cet énoncé, la documentation de [la librairie standard d'OCaml](#) (section [Input/output](#)) pourra vous être utile. Tous les fichiers ouverts en lecture seront supposés de contenu textuel.

Exercice 1 : Écrire un programme `my_cat.ml` dont l'exécutable effectuera le traitement suivant. La commande

```
my_cat <file1> [<file2>] ...
```

doit afficher dans le terminal le contenu des fichiers `<file1>`, `<file2>`, ...

Exercice 2 : Écrire un programme `get_line.ml` dont l'exécutable effectuera le traitement suivant. La commande

```
get_line <number> <input>
```

doit afficher le contenu de la ligne numéro `<number>` du fichier `<input>`, ou un message d'erreur si cette ligne n'existe pas.

Exercice 3 : Écrire un programme `fill_msg.ml` dont l'exécutable effectuera le traitement suivant. La commande

```
fill_msg <number> <message> <output>
```

doit créer le fichier `<output>` (ou écraser son contenu s'il existe déjà) en le remplissant par `<number>` lignes successives, chacune formée des caractères de `<message>`. Noter que le message peut contenir des espaces, à condition de l'écrire dans le terminal en l'encadrant par des guillemets (simples ou doubles, peu importe).

Exercice 4 : Écrire un programme `insert_line.ml` dont l'exécutable effectuera le traitement suivant. La commande

```
insert_line <number> <line> <input> <output>
```

doit copier le contenu du fichier `<input>` dans le fichier `<output>` en insérant immédiatement avant la ligne de numéro `<number>`, ou en fin de fichier si une telle ligne n'existe pas, une ligne supplémentaire formée des caractères de `<line>`.

Exercice 5 : Pour cet exercice, vous aurez besoin de la documentation du module [Random](#), de la fonction `Printf.fprintf` (module [Printf](#)) ainsi que, pour la seconde question, de la documentation des modules [Scanf](#) et [Scanf.Scanning](#)

1. Écrire un programme `fill_rand.ml` dont l'exécutable effectuera le traitement suivant. La commande

```
fill_rand <number> <sup> <output>
```

doit créer le fichier `<output>` (ou écraser son contenu s'il existe déjà) en le remplissant par `<number>` valeurs entières tirées aléatoirement dans l'intervalle `[0, ..., sup[`, écrites sous forme textuelle et séparées (ou suivies, peu importe) par des espaces.

2. Écrire un programme `sum.ml` dont l'exécutable effectuera le traitement suivant.

En supposant que le fichier `<input>` contient une suite de valeurs entières séparées (la première éventuellement précédée, la dernière éventuellement suivie) par des caractères d'espacement, par exemple un fichier généré par la commande précédente, la commande

```
sum <input>
```

doit afficher dans le terminal la suite des valeurs du fichier, ainsi que la somme de ces valeurs.

Exercice 6 : Améliorer le programme `my_cat.ml` de l'exercice 1 de la manière suivante : si le premier argument de la commande `my_cat` est l'option `-n`, chaque ligne affichée sera précédée de son numéro dans le fichier correspondant, les lignes étant numérotées à partir de 0 (on ne vous demande pas ici de vous servir du module `Arg`, cf. l'exercice suivant).

Exercice 7 : À partir de la documentation du module `Arg`, trouver le moyen d'écrire une seconde version du programme de l'exercice 3 dont la commande aura cette fois la forme suivante :

```
fill_msg -n <number> -m <message> -o <output>
```

Les options suivies de leurs valeurs pourront alors être écrites dans n'importe quel ordre.

On pourra tenter l'optimisation suivante : renvoyer un message d'erreur avant d'afficher l'usage de la commande si `<number>` est négatif, ou encore si la commande contient des arguments anonymes (cf. `Arg.Bad`)